

SQL & PostgreSQL

Fundamentos e Modelagem Relacional

Apostila de Estudos

Trilha Full Stack | Banco de Dados

Junho de 2026

PostgreSQL 18 SGBD utilizado	Ubuntu 26.04 LTS Ambiente de execucao	DBeaver 26 Interface visual	SSH Tunnel Acesso seguro
--	---	---------------------------------------	------------------------------------

Sumario

1. Introducao ao SQL

- O que e SQL
- Categorias de comandos (DDL, DML, DCL, TCL)
- Tipos de dados no PostgreSQL
- CRUD na pratica
- Constraints

2. O que e um Banco de Dados

- Definicao e conceitos
- Banco vs SGBD
- Relacionais vs NoSQL
- Propriedades ACID
- SQLite vs PostgreSQL

3. Diferencas entre Bancos Populares

- PostgreSQL
- MySQL / MariaDB
- SQLite
- MongoDB, Redis, Cassandra
- Comparativo direto

4. PostgreSQL em Profundidade

- Historia e filosofia
- Arquitetura interna e MVCC
- Estrutura de objetos
- pg_hba.conf e autenticao
- Roles e permissoes
- Comandos psql essenciais

5. Ambiente de Estudo

- Stack montada
- SSH Tunnel com chave ed25519
- Por que SSH Tunnel em vez de expor a porta
- Banco estudodb e role devuser

6. DBeaver — Interface Visual

- Navegacao
- SQL Editor
- Atalhos
- Explain Plan

7. Modelagem Relacional

- Por que modelar antes de criar
- Entidade, Atributo, PK, FK
- Tipos de relacionamento (1:1, 1:N, N:N)
- Notacao Crow's Foot
- Tabelas pivot (N:N)

8. Schema de Blog (Estudo de Caso 1)

- Diagrama ER
- Decisões de modelagem
- SQL completo comentado

9. Schema de E-commerce (Estudo de Caso 2)

- Diagrama ER
- Decisões de modelagem
- SQL completo comentado

10. Decisões de Arquitetura

- ON DELETE CASCADE vs RESTRICT vs SET NULL
- Soft Delete com `is_active`
- `unit_price` histórico em `order_items`
- Convenções de nomenclatura
- Tipos de dados: escolha certa

11. Glossário

Introducao ao SQL

O que e SQL

SQL (Structured Query Language) e a linguagem padrao para interagir com bancos de dados relacionais. Ao contrario de linguagens de programacao imperativas (onde voce diz *como* fazer), SQL e **declarativa**: voce declara *o que quer* e o banco resolve o "como".

```
-- Exemplo declarativo: voce nao diz como buscar, so o que quer
SELECT nome, email FROM usuarios WHERE ativo = true ORDER BY nome;
```

Nota: SQL nao e case-sensitive para palavras reservadas. SELECT, select e Select sao equivalentes. Por convencao, palavras reservadas ficam em MAIUSCULO.

Categorias de Comandos

Os comandos SQL sao divididos em quatro categorias:

Categoria	Sigla	Funcao	Exemplos
Data Definition Language	DDL	Define a estrutura do banco	CREATE, ALTER, DROP
Data Manipulation Language	DML	Manipula os dados nas tabelas	INSERT, SELECT, UPDATE, DELETE
Data Control Language	DCL	Controla permissoes de acesso	GRANT, REVOKE
Transaction Control Language	TCL	Controla transacoes	COMMIT, ROLLBACK, BEGIN

No dia a dia como desenvolvedor, voce trabalha principalmente com **DDL** (criando a estrutura) e **DML** (manipulando dados).

Tipos de Dados no PostgreSQL

Escolher o tipo correto afeta performance, integridade e espaco em disco:

Tipo	Descricao	Quando usar
INTEGER / BIGINT	Numero inteiro (4 ou 8 bytes)	IDs, quantidades, contagens
NUMERIC(p, s)	Precisao exata	Valores monetarios, evita erros de ponto flutuante
FLOAT / REAL	Ponto flutuante aproximado	Calculos cientificos, nao use para dinheiro
VARCHAR(n)	Texto com limite de caracteres	Nome, email, status
TEXT	Texto sem limite	Descricoes longas, conteudo de posts

CHAR(n)	Texto de tamanho fixo	Codigos padronizados: UF (2), CEP (9)
BOOLEAN	Verdadeiro ou falso	Flags: is_active, is_primary
DATE	Apenas data	Data de nascimento, data de entrega
TIMESTAMPZ	Data + hora + timezone	Registros de auditoria, created_at
SERIAL	Inteiro auto-incrementado	Chaves primarias simples
UUID	Identificador universal unico	PKs expostas em APIs (mais seguro)
INET	Endereco IP (IPv4 ou IPv6)	Logs de acesso, seguranca
JSONB	JSON binario indexavel	Dados semiestruturados, configs
TEXT[]	Array de texto	Tags, listas de valores

CRUD na Pratica

CRUD (Create, Read, Update, Delete) sao as quatro operacoes fundamentais de qualquer banco de dados:

CREATE TABLE — Criando estrutura

```
CREATE TABLE usuarios (
  id          SERIAL PRIMARY KEY,          -- PK auto-incrementada
  nome        VARCHAR(100) NOT NULL,       -- campo obrigatorio
  email       VARCHAR(255) NOT NULL UNIQUE, -- sem duplicatas
  ativo       BOOLEAN      DEFAULT true,    -- valor padrao
  criado_em   TIMESTAMPTZ  DEFAULT NOW()    -- preenchido automaticamente
);
```

INSERT — Inserindo dados

```
-- Um registro
INSERT INTO usuarios (nome, email)
VALUES ('Willians', 'willians@email.com');

-- Multiplos registros de uma vez
INSERT INTO usuarios (nome, email)
VALUES
  ('Ana', 'ana@email.com'),
  ('Carlos', 'carlos@email.com');
```

SELECT — Consultando dados

```
-- Todos os campos
SELECT * FROM usuarios;

-- Campos especificos
SELECT nome, email FROM usuarios;

-- Com filtro
SELECT nome FROM usuarios WHERE ativo = true;

-- Ordenado e limitado
```

```
SELECT nome, criado_em
FROM usuarios
ORDER BY criado_em DESC
LIMIT 10;
```

UPDATE — Atualizando dados

```
-- SEMPRE use WHERE em producao
UPDATE usuarios
SET ativo = false
WHERE email = 'carlos@email.com';
```

Atencao: UPDATE sem WHERE afeta TODOS os registros da tabela. Isso e irreversivel sem backup. Sempre confirme o WHERE antes de executar.

DELETE — Removendo dados

```
-- DELETE sem WHERE apaga TUDO – nunca faca isso em producao
DELETE FROM usuarios WHERE id = 3;

-- Alternativa segura: soft delete (nao apaga, so desativa)
UPDATE usuarios SET ativo = false WHERE id = 3;
```

Constraints

Constraints sao regras que o banco aplica automaticamente para garantir a integridade dos dados:

Constraint	O que garante	Exemplo
PRIMARY KEY	Unicidade e identidade do registro	id SERIAL PRIMARY KEY
NOT NULL	Campo obrigatorio	nome VARCHAR(100) NOT NULL
UNIQUE	Sem valores duplicados na coluna	email VARCHAR UNIQUE
DEFAULT	Valor automatico quando omitido	ativo BOOLEAN DEFAULT true
CHECK	Validacao logica customizada	preco NUMERIC CHECK (preco > 0)
FOREIGN KEY	Referencia valida a outra tabela	usuario_id REFERENCES usuarios(id)

O que é um Banco de Dados

Definicao

Um banco de dados é um **sistema organizado para armazenar, gerenciar e recuperar dados de forma eficiente e confiável**. Sem banco de dados, os dados vivem em arquivos soltos (CSV, JSON, TXT). Isso funciona para volumes pequenos, mas quebra em produção.

Banco de Dados vs SGBD

Dois conceitos que parecem iguais, mas são distintos:

Conceito	O que é	Exemplo
Banco de dados	O conjunto de dados organizado em disco	estudodb (o banco criado)
SGBD	O software que gerencia os bancos	PostgreSQL, MySQL, SQLite

Você não fala com o banco diretamente. Você fala com o SGBD (via psql, DBeaver ou aplicação), que por sua vez fala com os arquivos em disco.

Tipos de Banco de Dados

Relacionais (SQL)

Dados organizados em **tabelas** com linhas e colunas. Relacionamentos entre tabelas via chaves estrangeiras. Linguagem padrão: SQL.

Exemplos: **PostgreSQL**, MySQL, SQLite, SQL Server.

Não-Relacionais (NoSQL)

Dados em formatos alternativos, sem esquema fixo obrigatório. Cada tipo resolve um problema específico:

Tipo	Banco	Uso ideal
Documentos	MongoDB	Dados com estrutura variável, catálogos, CMS
Chave-Valor	Redis	Cache, sessões, filas de tarefas
Colunar	Cassandra	Séries temporais, logs em escala de bilhões
Grafos	Neo4j	Redes sociais, sistemas de recomendação

Propriedades ACID

Todo banco relacional sério garante **ACID** — o que o torna confiável para produção:

Letra	Propriedade	O que garante
-------	-------------	---------------

A	Atomicidade	Tudo acontece ou nada acontece. Sem estados intermediarios.
C	Consistencia	Os dados sempre ficam em estado valido apos uma transacao.
I	Isolamento	Transacoes concorrentes nao se interferem.
D	Durabilidade	Dados confirmados (COMMIT) nunca se perdem, mesmo com falha.

```
-- Exemplo ACID: transferencia bancaria
BEGIN;
    UPDATE contas SET saldo = saldo - 100 WHERE id = 1;
    UPDATE contas SET saldo = saldo + 100 WHERE id = 2;
COMMIT; -- os dois UPDATE sao confirmados juntos
-- Se qualquer um falhar: ROLLBACK automatico
-- Nunca fica com o saldo debitado e nao creditado
```

SQLite vs PostgreSQL

	SQLite	PostgreSQL
Uso ideal	Dev local, scripts, mobile	Producao, APIs, sistemas reais
Configuracao	Zero — e um arquivo .db	Precisa do servico rodando
Servidor	Nao tem	Processo postmaster
Concorrencia	Limitada (1 escrita)	Alta (MVCC)
Django default	Sim (startproject)	Troca de 3 linhas no settings.py

Diferenças entre Bancos Populares

PostgreSQL

O banco que está sendo estudado nesta apostila. Open source, desenvolvido desde 1986. Considerado o banco relacional mais avançado do mundo open source. Não é mantido por uma empresa, mas por uma comunidade global de engenheiros.

- ACID completo com MVCC (múltiplos leitores sem bloquear escritores)
- Suporte nativo a JSON/JSONB, arrays, UUID, INET, ranges
- Extensível: você pode criar tipos, funções e operadores próprios
- Ideal para: APIs, Django, sistemas web, produção

MySQL / MariaDB

Historicamente o banco padrão do ecossistema PHP/WordPress. MariaDB é o fork open source criado após a Oracle adquirir o MySQL.

- Rápido em leitura simples
- Dois engines: InnoDB (ACID) e MyISAM (sem transações, legado)
- Menos recursos avançados que o PostgreSQL
- Ideal para: projetos legados, WordPress, quando o cliente já usa MySQL

SQLite

Um banco que é um único arquivo .db, sem servidor, sem configuração. Django usa SQLite como padrão no startproject.

- Zero configuração — ideal para desenvolvimento local
- Concorrência muito limitada: uma escrita por vez
- Não é adequado para produção com múltiplos usuários simultâneos
- Ideal para: prototipagem, apps mobile, scripts, testes

MongoDB (NoSQL — Documentos)

Armazena documentos JSON (BSON internamente). Sem esquema fixo — cada documento pode ter campos diferentes.

```
// Sem CREATE TABLE. Inserção direta:
{
  "nome": "Willians",
  "email": "willians@email.com",
  "habilidades": ["Python", "Django", "React"],
  "endereco": { "cidade": "Ferraz de Vasconcelos" }
}
```

Faz sentido para dados com estrutura variável. Não faz sentido para dados com relacionamentos fortes (pedidos, clientes, produtos) — o PostgreSQL resolve melhor com JOINS.

Redis (NoSQL — Chave-Valor)

Banco em **memória**. Extremamente rápido. Não substitui o banco principal — complementa.

```
SET sessao:user123 "dados_da_sessao" # grava
GET sessao:user123 # le em < 1ms
EXPIRE sessao:user123 3600 # expira em 1 hora
```

- Cache de queries pesadas do PostgreSQL
- Sessoes de usuario (Django usa Redis por padrao em producao)
- Filas de tarefas (Celery + Redis)
- Rate limiting em APIs

Comparativo Direto

	PostgreSQL	MySQL	SQLite	MongoDB	Redis
Tipo	Relacional	Relacional	Relacional	Documento	Chave-Valor
ACID	Completo	InnoDB	Completo	Parcial	Nao
Servidor	Sim	Sim	Nao	Sim	Sim
JSON nativo	JSONB	Limitado	Nao	Nativo	Nao
Concorrencia	Alta	Alta	Baixa	Alta	Altissima
Com Django	Recomendado	Suportado	Default dev	Via pacote	Via pacote

Nota: Para a trilha Full Stack com Django: PostgreSQL para dados principais, Redis para cache e sessoes, SQLite apenas durante desenvolvimento local. Voce nao precisa dominar todos — PostgreSQL fundo te leva longe.

PostgreSQL em Profundidade

Historia e Filosofia

O PostgreSQL nasceu em 1986 na Universidade da California em Berkeley, como successor do projeto INGRES. Em 1996 o nome mudou para PostgreSQL (de "Post-INGRES + SQL"). Hoje é mantido pela **PostgreSQL Global Development Group** — uma comunidade de engenheiros distribuídos, sem dono corporativo.

Enquanto outros bancos apenas implementam o padrão SQL, o PostgreSQL **estende** o padrão com tipos exclusivos, funções e capacidades avançadas.

Tipos Exclusivos do PostgreSQL

```
-- Tipos que só o PostgreSQL tem nativamente
INET      -- endereços IP (IPv4 e IPv6)
CIDR      -- blocos de rede
JSONB     -- JSON binário, indexável
TEXT[]    -- array de qualquer tipo
UUID      -- identificador universal
RANGE     -- intervalos (ex: período de datas)
TSVECTOR  -- busca full-text nativa

-- Exemplo prático: armazenar IPs bloqueados (contexto Blue Team)
CREATE TABLE ips_bloqueados (
    id          SERIAL PRIMARY KEY,
    endereco    INET NOT NULL,
    motivo      TEXT,
    bloqueado_em TIMESTAMPTZ DEFAULT NOW()
);

INSERT INTO ips_bloqueados (endereco, motivo)
VALUES ('192.168.0.200', 'brute-force SSH detectado');
```

Arquitetura Interna

Entender a arquitetura ajuda a diagnosticar problemas e otimizar performance:

```
Instância PostgreSQL (cluster)
|
+-- Processo principal: postmaster
|   +-- Gerencia conexões de clientes
|   +-- Spawna processos filhos (um por cliente)
|   +-- Controla autenticação via pg_hba.conf
|
+-- Memória compartilhada
|   +-- shared_buffers (cache de páginas de dados)
|   +-- wal_buffers    (buffer de escrita antes do disco)
|
+-- WAL (Write-Ahead Log)
|   +-- Toda alteração é gravada AQUI antes de ir ao disco
|   +-- Garante o "D" do ACID (Durabilidade)
|   +-- Base para replicação e backup point-in-time
|
```

```
+-- Disco
+-- base/          (arquivos de dados reais)
+-- pg_wal/        (logs de transacao)
+-- pg_hba.conf    (regras de autenticacao)
```

MVCC — Controle de Concorrencia

Multi-Version Concurrency Control: leitores nunca bloqueiam escritores e escritores nunca bloqueiam leitores. Cada transacao ve um *snapshot* consistente do banco no momento em que iniciou.

Isso e critico em APIs com multiplos usuarios simultaneos — exatamente o cenario do stack Nginx + Unicorn + Django.

Estrutura de Objetos

```
Cluster (instancia na porta 5432)
|
+-- postgres          (banco do sistema – nao mexa)
+-- template1         (template para novos bancos)
+-- estudodb          (seu banco de estudo)
|
+-- Schema: public (padrao, onde suas tabelas ficam)
|   +-- Tabela: usuarios
|   +-- Tabela: posts
|   +-- View: relatorio_mensal
|
+-- Sequences (geram os IDs do SERIAL)
+-- Indexes   (aceleram queries)
+-- Functions (logica dentro do banco)
```

pg_hba.conf — Autenticacao

O arquivo que controla quem pode se conectar, de onde e como. Localizado em **/etc/postgresql/18/main/pg_hba.conf** no Ubuntu:

```
# Formato: TYPE DATABASE USER ADDRESS METHOD
local all postgres peer
local all all md5
host all all 127.0.0.1/32 scram-sha-256
host all all ::1/128 scram-sha-256

# peer = autentica pelo usuario do SO (sem senha na rede)
# md5 = senha com hash MD5
# scram-sha-256 = metodo mais seguro para conexoes TCP
```

Roles e Permissoes

O PostgreSQL nao tem "usuarios" — tem **roles**. Uma role pode ser um usuario (com LOGIN) ou um grupo.

```
-- Criar role para a aplicacao (nunca use o superuser postgres na app)
CREATE ROLE devuser WITH LOGIN PASSWORD 'senha_no_env';

-- Criar banco
CREATE DATABASE estudodb OWNER devuser;

-- Conceder apenas o necessario (principio do menor privilegio)
```

```
GRANT CONNECT ON DATABASE estudodb TO devuser;  
GRANT USAGE ON SCHEMA public TO devuser;  
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO devuser;
```

Nota: O mesmo principio do menor privilegio aplicado no systemd do Gunicorn e no socket Unix — a aplicacao nunca deve ter mais permissoes do que precisa.

Comandos Essenciais do psql

Comando	O que faz
\l	Listar todos os bancos
\c nome_banco	Conectar a um banco
\dt	Listar tabelas do schema atual
\d nome_tabela	Descrever estrutura de uma tabela (colunas, constraints)
\du	Listar roles e usuarios
\timing	Mostrar tempo de execucao das queries
\q	Sair do psql
\?	Ajuda sobre comandos do psql
\h SELECT	Ajuda sobre comando SQL especifico

Ambiente de Estudo

Stack Montada

Componente	Detalhe
Sistema Operacional	Ubuntu 26.04 LTS (instalado em SSD dedicado)
SGBD	PostgreSQL 18 (porta 5432, apenas localhost)
Interface visual	DBeaver 26 Community Edition (no Windows)
Acesso remoto	SSH Tunnel com chave ed25519
Banco de estudo	estudodb (role devuser, sem superuser)
IP do servidor	192.168.0.106 (rede local)

SSH Tunnel — Por que usar

O PostgreSQL por padrao escuta apenas em **localhost** (127.0.0.1). Isso e uma decisao de seguranca correta. Para acessar do Windows sem expor a porta 5432 na rede, usamos SSH Tunnel.

```
Windows (DBeaver)
|
| SSH Tunnel (porta 22, chave ed25519)
|
Ubuntu 192.168.0.106
|
| conexao local (localhost:5432)
|
PostgreSQL 18
|
estudodb (devuser)
```

- A porta 5432 NUNCA fica exposta na rede local
- A autenticao usa chave criptografica (ed25519), mais segura que senha
- A mesma chave usada para SSH normal serve para o tunnel do DBeaver
- Consistente com o principio do menor privilegio aplicado no restante da stack

Gerar e Configurar a Chave SSH

```
# No PowerShell do Windows:
ssh-keygen -t ed25519 -C "descricao-da-chave"
# Aceita o caminho padrao: C:\Users\usuario\.ssh\id_ed25519

# Ver a chave publica gerada:
type C:\Users\usuario\.ssh\id_ed25519.pub

# No Ubuntu – adicionar a chave publica:
mkdir -p ~/.ssh && chmod 700 ~/.ssh
echo "conteudo_da_chave_publica" >> ~/.ssh/authorized_keys
```

```
chmod 600 ~/.ssh/authorized_keys
```

```
# Testar a conexao por chave (deve conectar sem pedir senha):  
ssh -i C:\Users\usuario\.ssh\id_ed25519 willians@192.168.0.106
```

Configuracao no DBeaver

Aba	Campo	Valor
SSH Tunnel	Use SSH Tunnel	Marcado
SSH Tunnel	Host	192.168.0.106
SSH Tunnel	Port	22
SSH Tunnel	User	willians
SSH Tunnel	Authentication	Identity file
SSH Tunnel	Private key	C:\Users\willi\ssh\id_ed25519
Main	Host	localhost
Main	Port	5432
Main	Database	estudodb
Main	Username	devuser

DBeaver — Interface Visual

Mapa da Interface

```
+-----+
| Database Navigator (esquerda) |
| +-- estudodb                  |
|   +-- Schemas                |
|     +-- public                |
|       +-- Tables  <- suas tabelas |
|       +-- Views              |
|       +-- Sequences <- geradas pelo SERIAL |
+-----+
| SQL Editor (centro) | Properties (direita) |
| onde voce escreve   | detalhes do objeto  |
| queries             | selecionado          |
+-----+
| Results (abaixo)    |
| resultado das queries em grade editavel    |
+-----+
```

Atalhos Essenciais

Atalho	Acao
Ctrl +]	Abrir novo SQL Editor
Ctrl + Enter	Executar query sob o cursor
Ctrl + Shift + Enter	Executar todas as queries do editor
Ctrl + /	Comentar / descomentar linha
Ctrl + Shift + F	Formatar SQL automaticamente
Ctrl + Space	Autocomplete
F5	Refresh do Database Navigator
Ctrl + A	Selecionar tudo (util para executar script inteiro)

Funcionalidades Uteis

Ver estrutura de uma tabela

Navigator > Tables > nome_tabela > duplo clique:

- Aba "Columns" — colunas, tipos, constraints
- Aba "Constraints" — PK, UNIQUE, FK
- Aba "DDL" — SQL gerado para recriar a tabela (otimo para aprender)
- Aba "Data" — dados em grade, editavel diretamente

Explain Plan

Prefixe qualquer query com EXPLAIN ANALYZE para ver o plano de execucao:

```
EXPLAIN ANALYZE SELECT * FROM produtos WHERE preco > 500;

-- DBeaver exibe a aba "Execution Plan" com visualizacao grafica
-- Mostra o custo de cada operacao e se esta usando indices
```

Nota: Queries lentas sao vetor de ataque DoS em APIs. O Explain Plan ajuda a identificar table scans desnecessarios que deveriam usar um indice.

Exportar resultados

Apos um SELECT, clique com botao direito no resultado:

- Export Results > CSV (para planilhas)
- Export Results > JSON (para APIs)
- Export Results > SQL INSERT (para migrar dados)

Historico de queries

Menu > Window > SQL History — todas as queries executadas na sessao.

ER Diagram Automatico

O DBeaver gera automaticamente o diagrama ER do banco existente:

Navigator > estudodb > botao direito > View Diagram

Exibe todas as tabelas e relacionamentos detectados automaticamente. Util para verificar se as FKs foram criadas corretamente.

Modelagem Relacional

Por que Modelar Antes de Criar

Criar tabela diretamente no banco sem modelar antes é o erro mais comum. O custo de refatorar um banco em produção com dados é alto — exige migrations cuidadosas, possível downtime e risco de perda de dados.

O fluxo correto:

```
Requisitos de negocio
|
v
Modelagem (ERD no Lucidchart / dbdiagram.io)
|
v
Revisao e validacao
|
v
DDL (SQL) -> Banco de dados
```

Elementos de um Diagrama ER

Elemento	O que representa	No banco
Entidade	Uma "coisa" do mundo real com dados	Tabela
Atributo	Propriedade da entidade	Coluna
Chave Primaria (PK)	Identificador unico do registro	PRIMARY KEY
Chave Estrangeira (FK)	Referencia a outra tabela	FOREIGN KEY / REFERENCES
Relacionamento	Como entidades se conectam	JOIN entre tabelas
Cardinalidade	Quantos de cada lado	1:1, 1:N, N:N

Tipos de Relacionamento

1:1 — Um para Um

Cada registro de A se relaciona com no máximo um de B, e vice-versa.

```
usuarios      (1) <-----> (1) perfis
-- Um usuario tem um perfil. Um perfil pertence a um usuario.
-- A FK fica em perfis: usuario_id UNIQUE REFERENCES usuarios(id)
```

1:N — Um para Muitos

Um registro de A pode se relacionar com muitos de B. É o tipo mais comum. A FK sempre fica no lado "muitos".

```
usuarios      (1) <-----> (N) posts
```

```
-- Um usuario escreve muitos posts. Um post tem um unico autor.
-- A FK fica em posts: usuario_id REFERENCES usuarios(id)
```

N:N — Muitos para Muitos

Um registro de A pode se relacionar com muitos de B, e vice-versa. Exige uma **tabela pivot** (intermediaria).

```
posts      (N) <-----> (N) tags
-- Um post pode ter varias tags. Uma tag pode estar em varios posts.
-- Solucao: tabela pivot post_tags

CREATE TABLE post_tags (
    post_id INTEGER REFERENCES posts(id) ON DELETE CASCADE,
    tag_id  INTEGER REFERENCES tags(id)  ON DELETE CASCADE,
    PRIMARY KEY (post_id, tag_id) -- chave composta
);
```

Notacao Crow's Foot

A notacao padrao usada no Lucidchart e no DBeaver:

Simbolo	Significado
	Exatamente um (obrigatorio)
o	Zero ou um (opcional)
{	Um ou muitos
o{	Zero ou muitos
--o{	Um obrigatorio para zero ou muitos (1:N classico)
}o--o{	Zero ou muitos para zero ou muitos (N:N)

Ferramentas de Modelagem

Ferramenta	Tipo	Destaque
Lucidchart	Web (pago/freemium)	Interface visual completa, boa para apresentacoes
dbdiagram.io	Web (gratuito)	Escreve em texto, gera diagrama e exporta SQL
DBeaver	Desktop	Gera ERD automatico do banco existente
draw.io	Web/Desktop (gratis)	Offline, open source, versatil

Nota: Para estudar: use o Lucidchart para planejar antes, o DBeaver para verificar o que foi criado no banco. O dbdiagram.io e uma alternativa gratuita excelente ao Lucidchart.

Schema de Blog — Estudo de Caso 1

Análise de Requisitos

Antes de desenhar qualquer tabela, identificamos as entidades e regras:

- Usuarios escrevem posts (1 usuario -> N posts)
- Usuarios fazem comentarios em posts (1 usuario -> N comentarios)
- Posts recebem comentarios (1 post -> N comentarios)
- Posts podem ter varias tags, e uma tag pode estar em varios posts (N:N)

Diagrama ER



Decisões de Modelagem

Decisão	Motivo
usuarios tem email UNIQUE	Não pode ter dois usuarios com o mesmo email
posts.usuario_id FK	Post precisa de autor — FK garante que o autor existe
comentarios tem 2 FKs	Precisa do post e do autor — ambas obrigatorias
post_tags e tabela pivot	Implementa N:N sem duplicar dados
ON DELETE CASCADE em posts	Se o usuario for apagado, seus posts são apagados
ON DELETE CASCADE em comentarios	Se o post for apagado, comentarios vão junto

SQL Completo

```

-- Tabela base: sem dependencias
CREATE TABLE IF NOT EXISTS usuarios (
  id          SERIAL PRIMARY KEY,
  nome        VARCHAR(100) NOT NULL,
  email       VARCHAR(255) NOT NULL UNIQUE,
  ativo       BOOLEAN       DEFAULT true,
  criado_em   TIMESTAMPTZ   DEFAULT NOW()
);

-- Tabela independente: sem FK

```

```

CREATE TABLE IF NOT EXISTS tags (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(50) NOT NULL UNIQUE
);

-- Depende de: usuarios
CREATE TABLE IF NOT EXISTS posts (
    id SERIAL PRIMARY KEY,
    titulo VARCHAR(255) NOT NULL,
    conteudo TEXT NOT NULL,
    usuario_id INTEGER NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,
    criado_em TIMESTAMPTZ DEFAULT NOW()
);

-- Depende de: posts e usuarios
CREATE TABLE IF NOT EXISTS comentarios (
    id SERIAL PRIMARY KEY,
    texto TEXT NOT NULL,
    post_id INTEGER NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
    usuario_id INTEGER NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,
    criado_em TIMESTAMPTZ DEFAULT NOW()
);

-- Pivot N:N: posts <-> tags
-- PRIMARY KEY composta impede duplicatas (mesmo post, mesma tag)
CREATE TABLE IF NOT EXISTS post_tags (
    post_id INTEGER REFERENCES posts(id) ON DELETE CASCADE,
    tag_id INTEGER REFERENCES tags(id) ON DELETE CASCADE,
    PRIMARY KEY (post_id, tag_id)
);

```

Mapeamento para Django ORM

Quando o curso chegar nos models Django, cada entidade vira uma classe:

```

# models.py
class Usuario(models.Model):
    nome = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    ativo = models.BooleanField(default=True)
    criado_em = models.DateTimeField(auto_now_add=True)

class Tag(models.Model):
    nome = models.CharField(max_length=50, unique=True)

class Post(models.Model):
    titulo = models.CharField(max_length=255)
    conteudo = models.TextField()
    usuario = models.ForeignKey(Usuario, on_delete=models.CASCADE)
    tags = models.ManyToManyField(Tag) # pivot gerada automaticamente
    criado_em = models.DateTimeField(auto_now_add=True)

# Django ORM gera o SQL, FKs e a pivot post_tags automaticamente
# python manage.py makemigrations && python manage.py migrate

```

Schema de E-commerce — Estudo de Caso 2

Análise de Requisitos

Sistema de cadastro de clientes com produtos e pedidos:

- Clientes podem ter varios enderecos (entrega, cobranca) — 1:N
- Clientes podem ter varios telefones (celular, fixo, comercial) — 1:N
- Clientes fazem pedidos — 1:N
- Pedidos contem varios produtos, e produtos aparecem em varios pedidos — N:N via order_items
- Produtos sao entidades independentes (existem sem pedido)

Diagrama ER

```

CUSTOMERS
  |-- (1:N) --> ADDRESSES
  |-- (1:N) --> PHONE_NUMBERS
  |-- (1:N) --> ORDERS
  |
  +-- (1:N) --> ORDER_ITEMS <-- (N:1) -- PRODUCTS
  
```

SQL Completo

```

-- Entidades raiz: sem FKs
CREATE TABLE IF NOT EXISTS customers (
  id          SERIAL PRIMARY KEY,
  full_name   VARCHAR(100) NOT NULL,
  email       VARCHAR(255) NOT NULL UNIQUE,
  tax_id      VARCHAR(14)  NOT NULL UNIQUE,
  birth_date  DATE,
  is_active   BOOLEAN      DEFAULT true,
  created_at  TIMESTAMPTZ  DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS products (
  id          SERIAL PRIMARY KEY,
  name        VARCHAR(255)  NOT NULL,
  description  TEXT,
  price       NUMERIC(10, 2) NOT NULL,
  stock       INTEGER       DEFAULT 0,
  is_active   BOOLEAN      DEFAULT true,
  created_at  TIMESTAMPTZ  DEFAULT NOW()
);

-- CASCADE: endereco nao existe sem cliente
CREATE TABLE IF NOT EXISTS addresses (
  id          SERIAL PRIMARY KEY,
  customer_id INTEGER       NOT NULL REFERENCES customers(id) ON DELETE CASCADE,
  street      VARCHAR(255) NOT NULL,
  number      VARCHAR(10),
  complement  VARCHAR(100),
  neighborhood VARCHAR(100),
  city        VARCHAR(100) NOT NULL,

```

```

        state      CHAR(2)      NOT NULL,
        zip_code   VARCHAR(9)   NOT NULL,
        is_primary BOOLEAN      DEFAULT false
    );

-- CASCADE: telefone nao existe sem cliente
CREATE TABLE IF NOT EXISTS phone_numbers (
    id            SERIAL PRIMARY KEY,
    customer_id  INTEGER        NOT NULL REFERENCES customers(id) ON DELETE CASCADE,
    number       VARCHAR(15) NOT NULL,
    type        VARCHAR(20) DEFAULT 'mobile'
);

-- RESTRICT: nao pode apagar cliente com pedidos (historico fiscal)
CREATE TABLE IF NOT EXISTS orders (
    id            SERIAL PRIMARY KEY,
    customer_id  INTEGER        NOT NULL REFERENCES customers(id) ON DELETE RESTRICT,
    status       VARCHAR(20)    DEFAULT 'pending',
    total        NUMERIC(10, 2) DEFAULT 0,
    created_at   TIMESTAMPTZ    DEFAULT NOW()
);

-- Pivot N:N: orders <-> products
-- unit_price: preco no momento da compra (imutavel)
-- RESTRICT em product: nao pode apagar produto ja vendido
CREATE TABLE IF NOT EXISTS order_items (
    id            SERIAL PRIMARY KEY,
    order_id     INTEGER        NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
    product_id   INTEGER        NOT NULL REFERENCES products(id) ON DELETE RESTRICT,
    quantity     INTEGER        NOT NULL DEFAULT 1,
    unit_price   NUMERIC(10, 2) NOT NULL
);

```

Decisoes de Arquitetura

ON DELETE: CASCADE vs RESTRICT vs SET NULL

Quando um registro pai e apagado, o que acontece com os filhos? Essa e uma das decisoes mais importantes de modelagem:

Estrategia	Comportamento	Quando usar
CASCADE	Apaga os filhos automaticamente	Entidade nao faz sentido sem o pai Ex: enderecos, telefones, comentarios
RESTRICT	Bloqueia o DELETE se houver filhos	Historico tem valor independente Ex: pedidos de um cliente, itens de pedido
SET NULL	Define a FK como NULL nos filhos	Relacionamento opcional Ex: post sem autor apos moderacao
NO ACTION	Igual ao RESTRICT (comportamento padrao)	Quando voce quer o erro explicito

```
-- CASCADE: endereco vai junto com o cliente
customer_id REFERENCES customers(id) ON DELETE CASCADE

-- RESTRICT: nao deixa apagar cliente com pedidos
customer_id REFERENCES customers(id) ON DELETE RESTRICT

-- SET NULL: post fica sem autor (autor anonimizado)
usuario_id REFERENCES usuarios(id) ON DELETE SET NULL
```

Soft Delete com is_active

Em vez de usar DELETE para remover registros, desativamos com uma flag:

```
-- Hard delete (irreversivel, quebra FK se houver dependentes)
DELETE FROM customers WHERE id = 5;

-- Soft delete (seguro, historico preservado)
UPDATE customers SET is_active = false WHERE id = 5;

-- Consulta so os ativos
SELECT * FROM customers WHERE is_active = true;
```

Beneficios do soft delete:

- Preserva historico de pedidos, auditoria e rastreabilidade
- Nao quebra constraints de FK (pedidos ainda existem referenciando o cliente)
- Permite reativacao de registros
- Exigido em alguns contextos legais (LGPD, registros fiscais)

unit_price em order_items

Por que gravar o preço na tabela de itens do pedido em vez de só referenciar o produto?

```
-- order_items tem seu proprio unit_price
CREATE TABLE order_items (
    ...
    product_id INTEGER REFERENCES products(id),
    unit_price NUMERIC(10, 2) NOT NULL -- preço NO MOMENTO da compra
);

-- Se o produto mudar de preço depois:
UPDATE products SET price = 299.90 WHERE id = 7;

-- O historico de pedidos antigos permanece correto
-- order_items ainda tem unit_price = 199.90 (preço original)
```

Atencao: Se voce usasse só uma FK para products.price, qualquer alteracao de preço mudaria retroativamente o valor de todos os pedidos anteriores. Isso é invalido fiscalmente e gera relatorios incorretos.

Convenções de Nomenclatura

Portugues	Ingles	Motivo da escolha EN
clientes	customers	Padrao internacional, compativel com Django ORM
enderecos	addresses	Sem acento, plural correto em EN
telefonos	phone_numbers	Mais descritivo que "phones"
cpf	tax_id	Agnostico ao pais — funciona internacionalmente
ativo	is_active	Convencao booleana com prefixo is_
criado_em	created_at	Padrao universal em qualquer ORM
nome	full_name	Mais explicito sobre o que contem
logradouro	street	Equivalente semantico internacional
bairro	neighborhood	Equivalente semantico internacional
cep	zip_code	Agnostico ao pais

Escolha Certa do Tipo de Dado

Situacao	Tipo correto	Por que nao outro
Valores monetarios	NUMERIC(10,2)	FLOAT tem erro de ponto flutuante
Data de nascimento	DATE	TIMESTAMP inclui hora desnecessaria
Registro de eventos	TIMESTAMPTZ	TIMESTAMP sem TZ cria bugs em servidores UTC

Chave primaria simples	SERIAL	UUID e mais seguro em APIs expostas
Chave primaria em API	UUID	SERIAL e previsivel (enumeravel)
Codigo de estado (UF)	CHAR(2)	VARCHAR desperdicaria verificacao de tamanho
Descricao longa	TEXT	VARCHAR exige limite arbitrario
Status / tipo	VARCHAR(20)	TEXT e excessivo para valores curtos fixos
Endereco IP	INET	VARCHAR nao valida formato de IP

Glossario

ACID

Conjunto de propriedades que garantem confiabilidade em transações: Atomicidade, Consistência, Isolamento, Durabilidade.

CASCADE

Estratégia de FK: ao deletar o pai, os filhos são automaticamente deletados.

Cluster

Uma instância do PostgreSQL rodando. Pode conter vários bancos de dados.

Constraint

Regra de integridade aplicada automaticamente pelo banco (NOT NULL, UNIQUE, FK, etc).

CRUD

Create, Read, Update, Delete — as quatro operações fundamentais de qualquer banco.

DDL

Data Definition Language — comandos que criam e alteram a estrutura do banco (CREATE, ALTER, DROP).

DML

Data Manipulation Language — comandos que manipulam dados (INSERT, SELECT, UPDATE, DELETE).

ERD

Entity-Relationship Diagram — diagrama que representa visualmente entidades e relacionamentos.

FK

Foreign Key (Chave Estrangeira) — coluna que referencia a PK de outra tabela.

Hard Delete

Remoção permanente de um registro com DELETE. Irreversível.

INET

Tipo de dado exclusivo do PostgreSQL para armazenar endereços IP (IPv4 e IPv6).

JSONB

Tipo de dado exclusivo do PostgreSQL para armazenar JSON em formato binário, indexável.

LGPD

Lei Geral de Proteção de Dados — lei brasileira que regula o tratamento de dados pessoais.

MVCC

Multi-Version Concurrency Control — mecanismo do PostgreSQL onde leitores não bloqueiam escritores.

N:N

Relacionamento muitos-para-muitos. Exige tabela pivot para implementacao.

Notacao Crow's Foot

Padrao visual para representar cardinalidade em diagramas ER (||, o{, etc).

NULL

Ausencia de valor. Diferente de zero ou string vazia.

NUMERIC(p,s)

Tipo de dado com precisao exata. p = total de digitos, s = casas decimais. Ideal para dinheiro.

pg_hba.conf

Arquivo de configuracao do PostgreSQL que controla autenticacao de clientes.

PK

Primary Key (Chave Primaria) — identificador unico de um registro em uma tabela.

Postmaster

Processo principal do PostgreSQL que gerencia conexoes e spawna processos filhos.

RESTRICT

Estrategia de FK: bloqueia o DELETE do pai se houver filhos referenciando.

Role

Entidade de autenticacao no PostgreSQL. Pode ser um usuario (com LOGIN) ou um grupo.

Schema

Namespace dentro de um banco de dados. O schema "public" e o padrao.

SERIAL

Tipo atalho do PostgreSQL para INTEGER com sequencia auto-incrementada.

SET NULL

Estrategia de FK: ao deletar o pai, a FK nos filhos vira NULL.

SGBD

Sistema de Gerenciamento de Banco de Dados — software que gerencia os bancos (ex: PostgreSQL).

Soft Delete

Desativacao de registro com flag (is_active = false) em vez de DELETE fisico.

SQL

Structured Query Language — linguagem declarativa padrao para bancos relacionais.

SSH Tunnel

Tecnica de seguranca que encaminha uma conexao TCP por dentro de um canal SSH criptografado.

TIMESTAMPTZ

Tipo de data+hora com timezone. Preferível ao `TIMESTAMP` simples em sistemas distribuídos.

Transacao

Unidade atômica de trabalho no banco. Inicia com `BEGIN`, confirma com `COMMIT`, desfaz com `ROLLBACK`.

UUID

Universally Unique Identifier — identificador de 128 bits praticamente impossível de colidir.

WAL

Write-Ahead Log — mecanismo do PostgreSQL que grava alterações antes de ir ao disco, garantindo durabilidade.